

Part A: **Adapting** Software

1. Introduction
2. Evolution & Maintenance
3. Design Patterns
4. Architecture

Part B: **Verifying** Software

5. Compilation
6. Dynamic Analysis
7. Design by Contract
8. Configuration Management

Part C: **Modern** SE

9. **Open-Source Software**
10. AI for SE & SE for AI
11. Software Product Lines
12. Guest Lectures

9a. **Open-Source Software**

Open-Source Development
Central Questions for Each Software Project
Richard Stallman's Four Freedoms
Free Software
Buying Free Software
Summary

9b. **Open-Source Principles**

Rights and Duties
Collective Ownership
Recommendations for Companies
Copyleft and Copyright
The Impact of Licensing Issues
Summary

9c. **Open-Source Licenses**

MIT License
GPL: GNU General Public License
LGPL: GNU Lesser General Public License
Compatibility of Software Licenses
Re-Licensing and Dual Licensing
Creative Commons Licenses
Summary
FAQ

9. Open-Source Software – Handout

Software Engineering 2 | Chico Sundermann, Raphael Dunkel | June 22, 2026

9. Open-Source Software

9a. Open-Source Software

Open-Source Development

Central Questions for Each Software Project

Richard Stallman's Four Freedoms

Free Software

Buying Free Software

Summary

9b. Open-Source Principles

9c. Open-Source Licenses

Open-Source Development [Sommerville]

Open-Source Development

“**Open-source development** is an approach to software development in which the source code of a software system is published and volunteers are invited to participate in the development process.”

Open-Source Development

- assumption in the early days: code developed by small core group
- today: Internet to recruit volunteer developers (often users)
- volunteers contribute with bug reports, feature requests, changes to the software (cf. pull requests)
- core group controls changes to main repo
- opposite: proprietary software / closed-source development

Internet iff Open Source

- Internet used to distribute source code and recruit developers
- Internet builds on open-source software

Examples

- operating system Linux (for most servers)
- operating system Android (for most clients)
- web server Apache
- database management system MySQL
- browser Firefox
- image editor GIMP
- media player VLC
- screen recording software OBS
- video editing software Shotcut
- runtime env. and standard library OpenJDK
- development environment Eclipse

Central Questions for Each Software Project [Sommerville]

First Question

“Should the product that is being developed make use of open-source components?”

Example Criteria

- are related open-source components available?
- is their quality sufficient?
- will they be maintained?
- what is the effort to integrate them?
- is it cheaper to modify or redevelop them?
- how are they licensed?

Second Question

“Should an open-source approach be used for its own software development?”

Example Criteria

- business model: how to earn money then?
- support? consulting? proprietary extensions? – *rich relatives or friends? unconditional basic income (bedingungsloses Grundeinkommen)?*
- what additional value do you have from opening the source?
- more customers, recognition, collaborations?
- does it reveal confidential business knowledge?

Richard Stallman's Four Freedoms [Stallman's 2014 TEDx Talk]



Richard Matthew Stallman aka. RMS (born 1953)

Four Freedoms by Richard Stallman

- 0. freedom to run it for whatever purpose
- 1. freedom to study and modify the program (source code is available)
- 2. freedom to redistribute it (needed for non-programmers)
- 3. freedom to redistribute or even sell modified versions
- all four needed that the user controls the program
- otherwise the program controls the users

Free Software [Stallman's 2014 TEDx Talk]

Free Software

- free stands for freedom (often confused with “free of charge”)
- free software: demand for freedom
- open-source software: better code quality (promoted by Eric S. Raymond)
- today largely replaced with term open-source software
- more inclusive term?
free and open-source software (FOSS)

Free Software vs Freeware

- free software: free as in free speech
 - freeware: free as in free beer
- free software can be adware or shareware

Free Software Foundation

[fsf.org]

- founded by Richard Stallman in 1985
- first ten years: employ developers for the operating system GNU
Linux = GNU + Linux kernel
- later: promoting free software, working on legal issues



Buying Free Software



Open-Source Software – Summary

Lessons Learned

- Open-source development and open-source software
- Examples and central questions around open source
- Free software foundation: four freedoms, RMS, FOSS
- Next: What are main principles of open-source software?

Further Reading

- Sommerville, Chapter 7.4 Open-Source Development
- Stallman's 2014 TEDx Talk

Practice

1. Form groups of 2–3 students
2. Which open-source software are you using?
3. Have you ever made use of the four freedoms?

9. Open-Source Software

9a. Open-Source Software

9b. Open-Source Principles

- Rights and Duties

- Collective Ownership

- Recommendations for Companies

- Copyleft and Copyright

- The Impact of Licensing Issues

- Summary

9c. Open-Source Licenses

Rights and Duties

Who owns the code?

[Sommerville]

“Although a fundamental principle of open-source development is that source code should be freely available, this does not mean that anyone can do as they wish with that code. Legally, the developer of the code (either a company or an individual) owns the code. They can place restrictions on how it is used by including legally binding conditions in an **open-source software license**.”

What if there is no license?

- you are allowed to read the code
- you are **not allowed** to use, modify, distribute it
- you need to contact the owners to negotiate

Understanding Software Licenses

[fossa.com]

“Anyone who works with open-source software (OSS), whether as a developer, a contributor, or a business, has to know at least a little bit about open source licenses. In a nutshell, an open source license tells you what you can and can't do with the open source code. And if using the code comes with any requirements and/or responsibilities, the license outlines those as well.”

What's Next?

- principles of software licenses
- examples of famous software licenses

Collective Ownership



Sercan Leylek (2016)

[wordpress.com]

“Every programmer is an author.”

Mosher's Law of Software Engineering

[twitter.com]

“Don't worry if it doesn't work right. If everything did, you'd be out of a job.”

Recommendations for Companies

Motivation

- basically every company needs software
- even non-software companies need software
- more and more companies even transition into software companies (e.g., car manufacturers)
- large amounts of money spend on software licenses (e.g., Microsoft Windows and Office, Adobe Acrobat)
- even if open-source software is for free, resources are necessary to prevent license violations within a company
- and who does the maintenance? who pays for it?

Bayersdorfer 2007:

[Sommerville]

- track information about downloaded and used open-source components (e.g., storing licenses)
- understand how a component is licensed before it is used
- study the open-source project to predict its future evolution
- educate developers about open source and open-source licensing ✓
- auditing of open-source software to detect violations
- if you rely on open-source products, support their development

Copyleft and Copyright

Copyleft



- right to freely distribute and modify intellectual property
- obligation that the same rights (license) apply to derivative work
 - **weak copyleft**: only applies to formerly-licensed parts (e.g., a library)
 - **strong copyleft**: applies to the complete software (deprecatory: viral effect)
- license without copyleft is called **permissive**
- implemented with copyright laws

Examples for Copyleft Licenses

- GNU General Public License (GPL) for software
- Creative Commons share-alike license for documents and pictures



Copyright (Urheberrecht)

- kind of intellectual property, such as patents and trademarks (gestiges Eigentum, Patent, Marke)
- owner of creative work has exclusive right over copies for a limited time
- includes distribution, reproduction, public performance, derivative work
- copyright is often granted by national laws

Decision for copyleft is independent of:

- decision to allow/forbid commercial use
- decision about the fee to get the source code

The Impact of Licensing Issues [heise.de]

Ruby on Rails: Durch Lizenzproblem entfallene Library erzeugt Dominoeffekt

Eine halbe Million Open-Source-Projekte sind wohl von dem Chaos betroffen, das durch eine erst falsch lizenzierte, dann zurückgezogene Library entstanden ist.

Lesezeit: 5 Min.  In Pocket speichern

   376



March 2021: The Story of mimemagic

- Ruby library mimemagic is part of Ruby on Rails and was licensed under MIT license
- 580,000 repositories on Github use Ruby on Rails under the same license
- mimemagic is using another library under GPL license
- GPL has a copyleft: mimemagic must be published under GPL too
- old versions of mimemagic have been removed
- new versions use GPL license
- 580,000 projects would need to change from MIT to GPL and upgrade to the new version

Open-Source Principles – Summary

Lessons Learned

- Need for and impact of software licenses
- Ownership and code without any license
- Recommendations for companies
- Copyleft and its impact on software
- Next: What are major licenses?

Further Reading

- Sommerville, Chapter 7.4 Open-Source Development

Practice

1. Form groups of 2–3 students
2. Does published code without a license fulfill the four freedoms?
3. What is the problem of creating a license for every project?

9. Open-Source Software

9a. Open-Source Software

9b. Open-Source Principles

9c. Open-Source Licenses

- MIT License

- GPL: GNU General Public License

- LGPL: GNU Lesser General Public License

- Compatibility of Software Licenses

- Re-Licensing and Dual Licensing

- Creative Commons Licenses

- Summary

- FAQ

MIT License [fossa.com]

Profile

- released: about 1987
- publisher: Massachusetts Institute of Technology
- classification: permissive
- most popular license
- prominent use: Ruby on Rails (server-side web application framework), Node.js (JavaScript runtime environment)

Selected Terms and Conditions

- permits reuse within proprietary software
- license shipped with distribution or relicensing (even to proprietary licenses)
- “provided as-is, without warranty”



The Full License Text

“Copyright (c) <year> <copyright holders>

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.”

GPL: GNU General Public License [gnu.org]

Profile

- releases: 1989 (GPLv1), 1991 (GPLv2), 2007 (GPLv3)
- author/publisher: Richard Stallman, Free Software Foundation
- classification: strong copyleft
- prominent use: Linux kernel (GPL-2.0-only), GNU Compiler Collection (GCC)

Selected Terms and Conditions

- software using (modified) GPL software must be released under GPL (copyleft)
- for any sale or distribution: binaries must be accompanied by source code
- for private/interal use: no obligations
- commercial redistribution allowed
- GPL software (Linux/GCC) may be used for commercial purposes



Three Main Versions

- GPL-1.0-only and GPL-1.0-or-later
 - publishing binary only was allowed
 - applies to whole distributable
- GPL-2.0-only and GPL-2.0-or-later
 - fixed above problems
 - relaxed version LGPL, motivated by C standard library (libc)
- GPL-3.0-only and GPL-3.0-or-later
 - improved compatibility with other licenses (e.g., Apache)

LGPL: GNU Lesser General Public License [gnu.org]

Profile

- releases: 1991 (LGPLv2), 1999 (LGPLv2.1), 2007 (LGPLv3)
- formerly known as: GNU Library General Public License
- publisher: Free Software Foundation
- classification: weak copyleft

Selected Terms and Conditions

- similar to GPL, but less restrictive
- software using (modified) LGPL software components **does not have to** be released under LGPL
- modifications of LGPL software components **must** be released under LGPL
- freedom only for LGPL components



Three Main Versions

- LGPL-2.0-only and LGPL-2.0-or-later
- LGPL-2.1-only and LGPL-2.1-or-later
- LGPL-3.0-only and LGPL-3.0-or-later

What's the main problem of LGPL?

It contains "GPL"!

LGPL often confused with GPL in industry.

Compatibility of Software Licenses

Problem

- most licenses have similar goals
- still: sometimes legally impossible to combine software with different licenses
- incompatibilities often arise from copyleft clauses

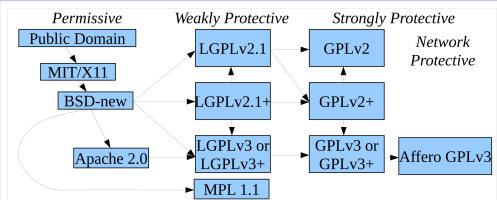
Desired Incompatibilities

- proprietary licenses are typically specific to a program and incompatible to each other
- copyleft licenses are incompatible to proprietary and permissive licenses
- software licensed with **GPL** cannot be used within **MIT**-licensed software

Undesired Incompatibilities

- software licensed with **GPL-2.0-only** cannot be used within **GPLv3-** or **LGPLv3**-licensed software (cf. Linux kernel) $\Rightarrow$ most projects use **GPL-2.0-or-later** instead
- software licensed with **GPLv3** cannot be used in material licensed as creative commons **BY-SA 4.0**
[creativecommons.org]

Overview on Compatibilities



Re-Licensing and Dual Licensing

How to solve a license incompatibility?

- remove the software everywhere
- re-implement (parts thereof)
- give it a new license (as replacement or in addition to the old license)

Re-Licensing

- changing the license to another license
- requires consent by all developers
- in practice 100% often not feasible: 80–95%

Re-Licensing of the VLC Project

- removed from the Apple App Store
- re-licensed from GPLv2 to LGPLv2
- later re-licensed to Mozilla Public License



Dual Licensing

- owner is not bound to the own copyleft/license
- a software can have multiple licenses
- same rules as for re-licensing
- term also used for 3 and more licenses



Netscape



Mozilla's Firefox Browser

- Netscape's Communicator 4.0 released under the Mozilla Public License
- incompatibility to GPL
- dual licensing as MPLv1.1/GPLv2/LGPLv2.1

Creative Commons Licenses – Overview

Concepts

- **Attribution (BY)**: Attribute the creator
- **Non-Commercial (NC)**: No commercial usage
- **Share-Alike (SA)**: Adaptations require same licensing terms
- **No-Derivatives (ND)**: Permits adaptations

CC Zero (CC0)

- Waive copyright
- Free use for any purposes
- No attribution required

CC in Software

Generally discouraged for software but may be useful for mixed projects (e.g., icons in GUI)

Creative Commons Licenses

★ Creative Commons provide a license that allows creators to share their work.



→ Attribution



→ Share-alike



→ Non-commercial



→ No Derivatives



- Credit required, anything allowed



- Credit, share alike always



- Credit, no changes allowed



- Credit, non-commercial use only



- Credit, noncommercial, share alike.



- Requires attribution, non-commercial, no derivatives.

Creative Commons Licenses – Intra Compatibility

	 PUBLIC DOMAIN	 PUBLIC DOMAIN	 BY	 BY SA	 BY NC	 BY ND	 BY NC SA	 BY NC ND
 PUBLIC DOMAIN	✓	✓	✓	✓	✓	✗	✓	✗
 PUBLIC DOMAIN	✓	✓	✓	✓	✓	✗	✓	✗
 BY	✓	✓	✓	✓	✓	✗	✓	✗
 BY SA	✓	✓	✓	✓	✗	✗	✗	✗
 BY NC	✓	✓	✓	✗	✓	✗	✓	✗
 BY ND	✗	✗	✗	✗	✗	✗	✗	✗
 BY NC SA	✓	✓	✓	✗	✓	✗	✓	✗
 BY NC ND	✗	✗	✗	✗	✗	✗	✗	✗

Creative Commons Licenses – Share Alike Compatibility

General

- Non-CC share alike licenses generally not compatible with BY-SA

One-Way Compatibility $X \rightarrow Y$

- Existing work under X **can** be adapted with changes under Y
- Existing work under Y **cannot** be adapted with changes under X
- Example: Work under **BY** can be adapted under **BY-SA** but not vice versa

ShareAlike Compatibility Process

- Ensure one-way/two-way compatibility between BY-SA and available relevant copyleft licenses
- Examine compatibility and adapt if necessary
- Currently compatible
 - Free Art License v1.3
 - GPLv3 (BY-SA $\rightarrow$ GPLv3)

Open-Source Licenses – Summary

Lessons Learned

- Software licenses: MIT, GPL, LGPL
- License incompatibilities: desired and undesired
- Re-licensing and dual licensing
- Create commons licenses: terms, (in)compatibilities
- Next: How to systematically reuse software?

Further Reading

- Sommerville, Chapter 7.4 Open-Source Development

Practice

Quiz in Stud.IP



FAQ – 9. Open-Source Software

Lecture 9a

- What characterizes open-source development?
- What are examples for successful open-source projects?
- What are the central questions on the usage of open-source software in a software project?
- What are Stallman's four freedoms?
- What is free software and how does it differ to freeware?
- What is the Free Software Foundation?

Lecture 9b

- With whom lies code ownership and why does this necessitate software licenses?
- What should companies keep in mind when using open-source software?
- What can companies do to keep used open-source projects alive?
- What is the difference between copyleft and copyright?
- What can mistakes on software licenses cause?

Lecture 9c

- What are examples for common open-source licenses?
- Which restrictions does the MIT license have?
- What are the GPL and LGPL licenses and how are they related?
- Why are some licenses incompatible and others not?
- Which license is usable in which usage scenario?
- What is re-licensing and which requirements does it have?
- What is dual-licensing (used for)?
- Which creative common licenses exist?
- How can CC licenses be combined with other licenses?